

Java Testing Advanced - Cheat Sheet

Once unit tests with JUnit and Mockito stop being enough, you reach for the heavier tools. Real databases in tests, HTTP stubbing, mutation testing, time travel, architecture rules.

Beyond unit tests

Unit tests catch logic bugs in one class. Real bugs often hide in the seams between classes, between services, or between your code and the infrastructure (DB, queues, HTTP). Advanced testing fills those gaps.

Concern	Tool
Spring app context	Spring Boot Test
Real DB, Kafka, Redis in tests	Testcontainers
HTTP service stubbing	WireMock
Async / eventual consistency	Awaitility
Time-dependent logic	Inject Clock
Property-based testing	jqwik
Mutation testing	PIT
Architecture rules	ArchUnit
Coverage	JaCoCo
Microbenchmarks	JMH

Spring Boot test slicing

Loading the entire Spring context for every test is slow. Slice annotations load only the parts you need.

```

// Full app context (slow but realistic)
@SpringBootTest
class FullIntegrationTest {
    // tests live here
}

// Only the web layer
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired
    MockMvc mvc;

    @MockBean
    UserService service;

    @Test
    void returns200OnFoundUser() throws Exception {
        when(service.find(42L)).thenReturn(Optional.of(new User(42L, "alice")));

        mvc.perform(get("/users/42"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("alice"));
    }
}

// Only JPA repositories with an embedded DB
@DataJpaTest
class UserRepositoryTest {
    @Autowired
    UserRepository repo;

    @Test
    void persistsAndLoadsUser() {
        repo.save(new User("alice"));
        assertThat(repo.findByName("alice")).isPresent();
    }
}

```

Common slice annotations:

Annotation	Loads
@SpringBootTest	Full app context
@WebMvcTest	MVC controllers, no service or repository layer
@DataJpaTest	JPA repositories + embedded DB
@JsonTest	JSON serialization (Jackson, Gson)
@RestClientTest	RestTemplate / WebClient infrastructure
@DataMongoTest	Spring Data MongoDB

Slices keep tests fast by skipping beans you don't need.

Testcontainers (real DB in tests)

H2 and embedded DBs are convenient but lie about behavior (different SQL dialect, different transaction semantics). Testcontainers spins up real Postgres, MySQL, Kafka, or Redis in Docker for the duration of your tests.

```

@SpringBootTest
@Testcontainers
class OrderServiceIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16")
        .withDatabaseName("test")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource
    static void postgresProps(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    OrderRepository repo;

    @Test
    void persistsOrder() {
        Order order = new Order("alice@example.com", 100.0);
        repo.save(order);
        assertThat(repo.count()).isEqualTo(1);
    }
}

```

Containers can be shared across the test class (`static`) or recreated per test. Sharing is faster. Recreating gives you isolation.

Common containers: `PostgreSQLContainer` , `MySQLContainer` , `KafkaContainer` , `RedisContainer` , and `GenericContainer` for anything else.

WireMock (HTTP stubbing)

When your code talks to an external HTTP service, WireMock stands in as a fake server.

```
@SpringBootTest
@AutoConfigureWireMock(port = 0)
class PaymentClientTest {

    @Autowired
    PaymentClient client;

    @Value("${wiremock.server.port}")
    int wiremockPort;

    @Test
    void parsesSuccessfulCharge() {
        stubFor(post(urlEqualTo("/charges"))
            .willReturn(aResponse()
                .withStatus(200)
                .withHeader("Content-Type", "application/json")
                .withBody("{\"id\":\"ch_123\",\"status\":\"succeeded\"}")));

        PaymentResult result = client.charge(100.0);

        assertThat(result.getId()).isEqualTo("ch_123");
        verify(postRequestedFor(urlEqualTo("/charges"))
            .withRequestBody(matchingJsonPath("$.amount", equalTo("100.0"))));
    }
}
```

Useful patterns:

```
// Conditional response based on body
stubFor(post("/charges")
    .withRequestBody(matchingJsonPath("$.amount", equalTo("1000")))
    .willReturn(aResponse().withStatus(402))); // payment required

// Network delay simulation
stubFor(get("/slow").willReturn(aResponse().withFixedDelay(2000)));

// Fault simulation
stubFor(get("/flaky").willReturn(aResponse().withFault(Fault.CONNECTION_RESET_BY_PEER)));
```

WireMock can also record real HTTP traffic and replay it, useful for capturing edge cases from a live service.

Mockito advanced

BDD-style

`org.mockito.BDDMockito` renames the methods to match given/when/then:

```
given(repo.findById(42L)).willReturn(Optional.of(user));

service.find(42L);

then(repo).should().findById(42L);
```

Functionally identical to standard Mockito. Easier to read for BDD-style tests.

Spies (partial mocks)

A spy wraps a real object and lets you override specific methods.

```
List<String> list = new ArrayList<>();
List<String> spy = spy(list);

spy.add("real");           // adds to the real list
when(spy.size()).thenReturn(100); // override size

assertEquals(100, spy.size()); // returns 100
assertEquals("real", spy.get(0)); // real call
```

Use spies when most of an object's behavior is correct and you only want to fake one method. They're a smell more often than not.

Static methods

Since Mockito 3.4:

```
try (MockedStatic<UUID> mocked = mockStatic(UUID.class)) {
    mocked.when(UUID::randomUUID).thenReturn(
        UUID.fromString("00000000-0000-0000-0000-000000000001"));

    UUID id = UUID.randomUUID();
    assertEquals("00000000-0000-0000-0000-000000000001", id.toString());
}
```

The mock applies only inside the try-with-resources block. Use sparingly.

Final classes and methods

Add `mockito-inline` to your dependencies. Then `mock(FinalClass.class)` works.

Strict stubbing

Mockito 4+ defaults to strict mode, which fails the test if you stub something the code never calls. Switch to lenient per stub:

```
lenient().when(repo.findById(any())).thenReturn(Optional.empty());
```

Or set it for the whole class with `@MockitoSettings(strictness = Strictness.LENIENT)`.

Test data builders

Constructors with 10 parameters make tests ugly. Builders fix it.

```
public class UserBuilder {
    private String name = "default";
    private int age = 30;
    private String email = "default@example.com";
    private boolean active = true;

    public static UserBuilder aUser() {
        return new UserBuilder();
    }

    public UserBuilder withName(String name) {
        this.name = name;
        return this;
    }

    public UserBuilder withAge(int age) {
        this.age = age;
        return this;
    }

    public UserBuilder inactive() {
        this.active = false;
        return this;
    }

    public User build() {
        return new User(name, age, email, active);
    }
}

// In a test
User user = aUser().withName("alice").inactive().build();
```

Defaults handle the boring fields. Tests show only what matters for that case.

For codebases that don't want hand-written builders, Lombok's `@Builder` or libraries like Instancio (random test data) can fill the gap.

Custom AssertJ assertions

When the same multi-line assertions keep showing up, build a custom assertion class.

```
public class UserAssert extends AbstractAssert<UserAssert, User> {

    public UserAssert(User actual) {
        super(actual, UserAssert.class);
    }

    public static UserAssert assertThat(User actual) {
        return new UserAssert(actual);
    }

    public UserAssert isActive() {
        assertNotNull();
        if (!actual.isActive()) {
            failWithMessage("Expected user to be active but was inactive");
        }
        return this;
    }

    public UserAssert hasEmail(String email) {
        assertNotNull();
        if (!actual.getEmail().equals(email)) {
            failWithMessage("Expected email <%s> but was <%s>", email,
                actual.getEmail());
        }
        return this;
    }
}

// Usage
UserAssert.assertThat(user).isActive().hasEmail("alice@example.com");
```

Custom assertions give better failure messages and chain naturally in your domain language.

Async testing with Awaitility

Polling loops with `Thread.sleep` are flaky and slow. Awaitility waits for a condition with a clean API.

```
import static org.awaitility.Awaitility.await;

@Test
void eventuallyMarksAsProcessed() {
    service.startProcessing();

    await()
        .atMost(5, SECONDS)
        .pollInterval(100, MILLISECONDS)
        .until(() -> repo.findById(42L).get().isProcessed());
}
```

Tunables: `atMost`, `pollInterval`, `pollDelay`, `untilAsserted` (for AssertJ-style waits inside the `await`).

Keep timeouts realistic. Too short and tests flake. Too long and a real failure waits forever.

Time-dependent testing

Code that calls `Instant.now()` directly is hard to test. Inject a `Clock` instead.

```
public class Order {
    private final Clock clock;
    private Instant createdAt;

    public Order(Clock clock) {
        this.clock = clock;
        this.createdAt = clock.instant();
    }
}

// Production
new Order(Clock.systemUTC());

// Test
Clock fixed = Clock.fixed(Instant.parse("2025-01-01T12:00:00Z"), ZoneOffset.UTC);
Order order = new Order(fixed);
assertEquals(Instant.parse("2025-01-01T12:00:00Z"), order.getCreatedAt());
```

For legacy code you can't refactor, `Mockito.mockStatic(Instant.class)` works but feels like surgery on a moving patient.

Property-based testing (jqwik)

Generate random inputs and assert invariants. Catches edge cases human-written tests miss.


```

@property
boolean reverseTwiceIsIdentity(@ForAll List<Integer> list) {
    return reverse(reverse(list)).equals(list);
}

@property
boolean encodedThenDecodedReturnsOriginal(@ForAll String input) {
    return decode(encode(input)).equals(input);
}

```

jqwik generates many inputs (default 1000) and tries to shrink failing cases to the smallest example.

Good fits:

- Pure functions with mathematical properties.
- Serialization round-trips.
- Parser / formatter pairs.

Not so good for code with lots of state or side effects.

Mutation testing (PIT)

Coverage tools tell you which lines ran. Mutation testing asks the harder question: would your tests catch the bug if the code changed?

PIT modifies the bytecode (changes `<` to `<=`, `&&` to `||`, flips return values), reruns the tests, and reports which mutations the tests killed.

```
mvn org.pitest:pitest-maven:mutationCoverage
```

A 90% line coverage with 30% mutation coverage means most of your “tests” don’t actually verify behavior. Worth running on critical modules.

Slow on large codebases. Add it to a weekly job rather than every commit.

Architecture testing with ArchUnit

Enforce architectural rules as tests. Stops the codebase from drifting into chaos over time.

```

class ArchitectureTest {

    @Test
    void controllersOnlyDependOnServices() {
        ArchRuleDefinition.classes()
            .that().resideInAPackage("..controller..")
            .should().onlyDependOnClassesThat()
            .resideInAnyPackage(
                "..controller..", "..service..", "java..", "org.springframework..")
            .check(new ClassFileImporter().importPackages("com.example"));
    }

    @Test
    void noServiceDependsOnController() {
        ArchRuleDefinition.noClasses()
            .that().resideInAPackage("..service..")
            .should().dependOnClassesThat()
            .resideInAPackage("..controller..")
            .check(new ClassFileImporter().importPackages("com.example"));
    }
}

```

Common rules to enforce:

- Layers can only depend downward.
- Domain classes don't import Spring.
- No package cycles.
- Specific classes must be `final`.

Coverage with JaCoCo

Reports which lines and branches the tests executed.

```

<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>

```

Report lives at `target/site/jacoco/index.html` .

Aim for 80%+ on business logic. Chasing 100% pushes you to test getters, setters, and DI wiring just to inflate the number.

Handling flaky tests

A flaky test passes most of the time and fails occasionally. Worse than a broken test, because it teaches the team to ignore failures.

Common causes:

Cause	Fix
Time-dependent (<code>now()</code> , <code>LocalDate.now()</code>)	Inject <code>Clock</code>
Order-dependent	Don't share mutable state between tests
Race conditions in async code	Use <code>Awaitility</code> , never <code>Thread.sleep()</code>
Random data without fixed seed	Use a fixed seed or property-based tools
External services	Mock with <code>WireMock</code>
Resource leaks (port, file)	Ensure <code>@AfterEach</code> cleans up
Test interleaving	Disable parallel execution per class with <code>@Execution(SAME_THREAD)</code>

When you find a flaky test, fix it or delete it. Letting it linger erodes trust in the whole suite.

Performance testing with JMH

JMH (Java Microbenchmark Harness) runs microbenchmarks correctly. Naive timing with `System.currentTimeMillis()` ignores JVM warmup, JIT, and dead-code elimination, so the numbers are meaningless.

```
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 3, time = 1, timeUnit = TimeUnit.SECONDS)
@Measurement(iterations = 5, time = 1, timeUnit = TimeUnit.SECONDS)
public class StringConcatBenchmark {

    @Benchmark
    public String concatWithPlus() {
        String s = "";
        for (int i = 0; i < 100; i++) {
            s += i;
        }
        return s;
    }

    @Benchmark
    public String concatWithBuilder() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 100; i++) {
            sb.append(i);
        }
        return sb.toString();
    }
}
```

Don't put JMH benchmarks in the regular test suite. They take minutes and belong in a separate module or task.

Best practices

- Match the tool to the question. Unit tests for logic, Testcontainers for DB behavior, WireMock for HTTP, ArchUnit for layering rules.
- Keep slow tests in a separate suite. Run them only in CI.
- Fix flakes immediately. A 95% reliable test is worse than no test.
- Use builders for test data. Long inline constructors hide the point of the test.
- Inject `Clock` and `Random` for testability. Even if today's test doesn't need it, tomorrow's might.
- Test the contract your code exposes. Skip tests that assert framework wiring.
- Treat coverage as a signal. Chasing 100% with weak assertions doesn't make code better.

Common gotchas

- `@MockBean` resets the Spring context. Spring caches contexts across tests, but `@MockBean` invalidates the cache. Tests slow down dramatically if you mix many `@MockBean` variants.
- Testcontainers slow startup. Cache Docker images. Reuse containers across tests with `withReuse(true)` (requires `testcontainers.reuse.enable=true` in `~/.testcontainers.properties`).
- Forgetting to close `MockedStatic`. Leaks the mock across tests. Always use `try-with-resources`.
- Sharing mutable test data builders. Make builders return new instances on each call, or chain methods can mutate shared state.
- Mocking the framework. Don't mock Spring, JPA, or Hibernate internals. Test against the real thing or use slices.
- Over-verifying with Mockito. Tight verification makes refactoring painful. Verify essential interactions only.
- Awaitility with no timeout. Default is a sane value, but always set `atMost` explicitly so failures fail fast.
- JMH benchmarks in the test path. Slows the test suite to a crawl. Keep them isolated.

Quick reference

Tool	Purpose
Spring Boot Test slices	Load only the layer you need
Testcontainers	Real DB / Kafka / Redis in tests via Docker
WireMock	Stub external HTTP services
BDDMockito	given/when/then-style mocking
@Spy	Wrap a real object, override specific methods
MockedStatic	Mock static methods (try-with-resources)
Test Data Builder	Readable test object creation
Custom AssertJ	Chainable domain-specific assertions
Awaitility	Wait for async conditions cleanly
Clock injection	Deterministic time-dependent tests
jqwik	Property-based testing
PIT	Mutation testing
ArchUnit	Architecture rules as tests
JaCoCo	Coverage report
JMH	Accurate microbenchmarks